

CHAPTER 06

메서드

- 01 메서드 기본 형태
- 02 매개변수와 반환
- 03 클래스 메서드
- 04 오버로딩
- 05 접근 제한자
- 06 생성자
- 07 속성
- 08 값 복사와 참조 복사
- 09 함께하는 응용예제
- 10 원도 폼에서 메서드 활용하기

01 메서드 기본 형태

- 메서드의 기본적인 형태는 다음과 같음

```
[접근제한자] [반환형] [메서드 이름]([매개변수])  
{  
    _____ 이후에 자세히 알아볼 것이므로 일단 public을 입력한다고 생각합시다.  
    [메서드 코드]  
}
```

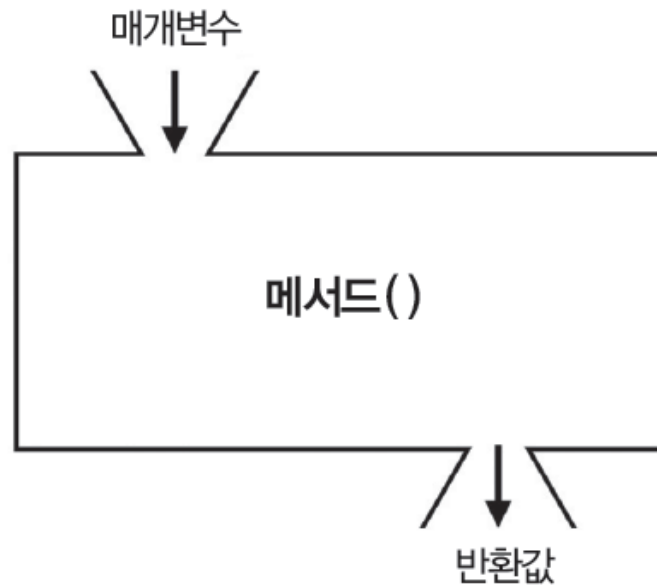


그림 6-1 함수 상자와 메서드

01 메서드 기본 형태

- 매개변수를 넣어주면, 메서드 내부에서 특정한 처리를 수행하고 메서드 내부에서 만들어진 값을 반환함

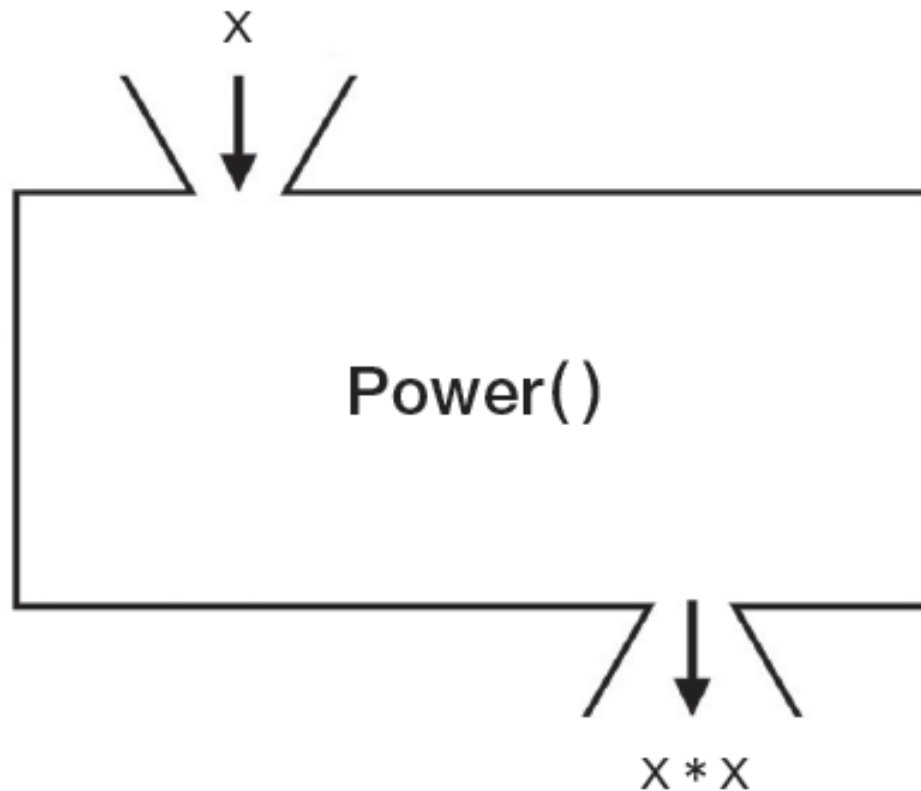


그림 6-2 `Power()` 메서드

01 메서드 기본 형태

■ 기본예제 6-1 인스턴스 메서드 생성과 사용

/6장/InstanceMethod

- 다음 코드는 Test 클래스에 매개변수로 넣은 숫자를 제공하는 메서드 Power를 생성

코드 6-1 인스턴스 메서드 생성과 사용

```
01 class Program
02 {
03     class Test
04     {
05         public int Power(int x)
06         {
07             return x * x;
08         }
09     }
10
11     static void Main(string[] args)
12     {
13         Test test = new Test();
14         Console.WriteLine(test.Power(10));
15         Console.WriteLine(test.Power(20));
16     }
17 }
```

실행 결과

100

400

01 메서드 기본 형태

- 메서드는 매개변수를 여러 개 가질 수도 있음

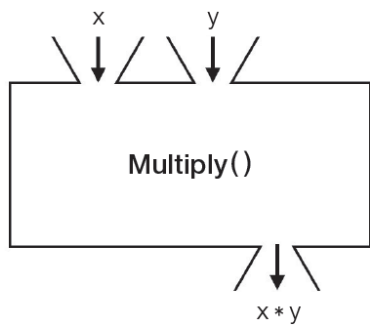


그림 6-3 Multiply() 메서드

코드 6-2

두 개의 매개변수를 갖는 메서드

/6장/Methods

```
01 class Program
02 {
03     class Test
04     {
05         public int Multi(int x, int y)
06         {
07             return x * y;
08         }
09     }
10
11     static void Main(string[] args)
12     {
13         Test test = new Test();
14         Console.WriteLine(test.Multi(52, 273));
15         Console.WriteLine(test.Multi(103, 32));
16     }
17 }
```



Microsoft Visual

14196

3296

01 메서드 기본 형태

- 메서드는 아무것도 반환하지 않아도 됨
- 아무것도 반환하지 않을 때는 [반환형] 위치에 void를 입력

Print()

그림 6-4 Print() 메서드

코드 6-3

아무것도 반환하지 않는 메서드

/6장/Methods

```
01 class Program
02 {
03     class Test
04     {
05         public void Print()
06         {
07             Console.WriteLine("Print() 메서드가 호출되었습니다.");
08         }
09     }
10
11
12     static void Main(string[] args)
13     {
14         Test test = new Test();
15         test.Print();
16         test.Print();
17         test.Print();
18     }
19 }
```

Microsoft Visual Studio 디버깅

Print() 메서드가 호출되었습니다.
Print() 메서드가 호출되었습니다.
Print() 메서드가 호출되었습니다.

02 매개변수와 변환

- 일반적으로 무언가를 반환하는 메서드는 다음과 같은 형태

```
public 자료형 메서드(자료형 매개변수, 자료형 매개변수)
{
    자료형 output = 초깃값;

    // output에 값을 계산

    return output;
}
```

02 매개변수와 변환

■ 기본예제 6-2 매개변수와 반환(1)

/6장/SumMethod

- 다음 코드는 min부터 max까지의 숫자를 더해 반환하는 메서드를 생성하고 호출

코드 6-4 매개변수와 반환(1)

```
01 class Program
02 {
03     class Test
04     {
05         public int Sum(int min, int max)
06         {
07             int output = 0;
08             for (int i = min; i <= max; i++)
09             {
10                 output += i;
11             }
12             return output;
13         }
14     }
15
16     static void Main(string[] args)
17     {
18         Test test = new Test();
19         Console.WriteLine(test.Sum(1, 100));
20     }
21 }
```

실행 결과

5050

02 매개변수와 변환

■ 기본예제 6-3 매개변수와 반환(2)

/6장/MultiplyMethod

- 다음 코드는 min부터 max까지의 숫자를 곱해 반환하는 메서드를 생성하고 호출

코드 6-5 매개변수와 반환(2)

```
01 class Program
02 {
03     class Test
04     {
05         public int Multiply(int min, int max)
06         {
07             int output = 1;
08             for (int i = min; i <= max; i++)
09             {
10                 output *= i;
11             }
12             return output;
13         }
14     }
15
16
17     static void Main(string[] args)
18     {
19         Test test = new Test();
20         Console.WriteLine(test.Multiply(1, 10));
21     }
22 }
```

실행 결과

3628800

03 클래스 메서드

- 메서드도 **인스턴스 메서드** 와 **클래스 메서드** 가 있음

1. 인스턴스 메서드 (Instance Method)

*객체(인스턴스)를 생성한 후에만 호출할 수 있는 메서드입니다.

*특징: 각 객체의 개별적인 상태(인스턴스 변수)를 다룰 때 사용합니다.

*호출 방법: 객체이름.메서드이름() 형식으로 호출합니다.

*예: 자동차 객체가 있다면, 각 자동차마다 속도가 다르므로
Accelerate() 같은 메서드는 인스턴스 메서드가 적합합니다.

2. 클래스 메서드 (Static Method)

- *객체를 생성하지 않고도 클래스 이름을 통해 바로 호출할 수 있는 메서드입니다.
C#에서는 `static` 키워드를 붙여 정의합니다.
- *특징: 특정 객체의 상태에 의존하지 않고, 공통적인 기능을 수행할 때 사용합니다.
- *호출 방법: 클래스이름.메서드이름() 형식으로 호출합니다.
- *예: `Math.Abs()`(절댓값 계산)처럼 입력값만 있으면 결과가 나오는 유틸리티 함수들이 대표적입니다.

03 클래스 매서드

구분	인스턴스 메서드	클래스(Static) 메서드
선언 방식	일반적인 선언	static 키워드 사용
메모리 할당	객체가 생성될 때 할당	프로그램 시작 시 할당
접근 대상	인스턴스 변수 & static 변수 모두 가능	static 변수만 가능
호출 예시	<code>myCar.Drive();</code>	<code>Car.GetTotalCount();</code>

03 클래스 메서드

- 기본 콘솔 프로젝트로 생성되는 Program 클래스를 보면 Main() 메서드가 있음

코드 6-6

Main() 메서드는 클래스 메서드

/6장/Methods

```
01 class Program
02 {
03     static void Main(string[] args)
04     {
05
06     }
07 }
```

03 클래스 메서드

■ 클래스 메서드 생성 방법

```
[접근 제한자] static [반환형] [메서드 이름]([매개변수])  
{  
  
}
```

- Program 클래스 내부에 있는 Main() 메서드를 직접 사용하고 싶다면,
클래스 이름 뒤에 . 기호를 찍고 곧바로 사용

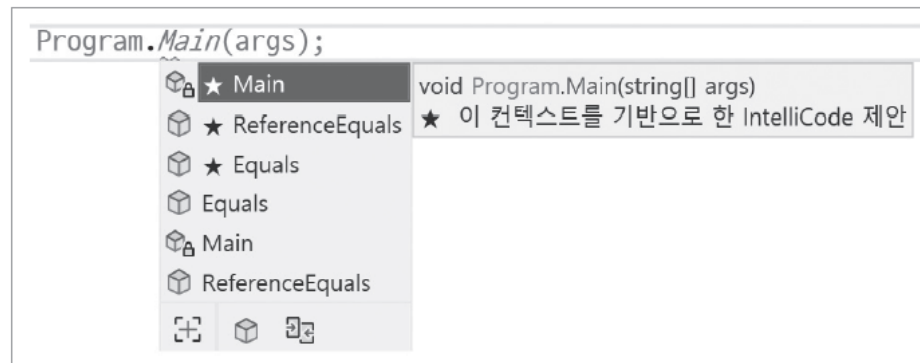


그림 6-5 Main() 메서드

03 클래스 메서드

■ 기본예제 6-4 클래스 메서드 생성과 사용

/6장/ClassMethod

- 다음 코드는 클래스 메서드 MyMath.Abs()를 만들고 호출

코드 6-7 메서드 생성과 사용

```
01 class Program
02 {
03     class MyMath
04     {
05         public static int Abs(int input)
06         {
07             if (input < 0)
08             {
09                 return -input;
10             }
11             else
12             {
13                 return input;
14             }
15         }
16     }
17
18     static void Main(string[] args)
19     {
20         Console.WriteLine(MyMath.Abs(52));
21         Console.WriteLine(MyMath.Abs(-273));
22     }
23 }
```

실행 결과

52
273

■ 클래스 메서드에서 사용할 수 있는 것

- static이라는 키워드를 붙인 변수 또는 메서드는 프로그램을 실행하는 순간에 메모리에 올라가게 됨
 - Math.Abs()와 같은 클래스 메서드는 인스턴스화해서 메모리에 올리는 별도의 작업(new Math()) 없이도 사용할 수 있음
 - 클래스 메서드에서는 아직 메모리에 올라가지 않은 인스턴스 변수와 인스턴스 메서드를 사용할 수 없음

코드 6-8

클래스 메서드에서 인스턴스 변수 사용은 오류가 발생

/6장/ClassMethods

```
01 class Program
02 {
03     public int instanceVariable = 10;
04
05     static void Main(string[] args)
06     {
07         Console.WriteLine(instanceVariable);
08     }
09 }
```


■ 클래스 메서드에서 사용할 수 있는 것

- 이를 수정하려면 사용하고자 하는 인스턴스 변수와 인스턴스 메서드에 static 키워드를 붙여, 클래스 변수와 클래스 메서드로 만들어줘야 함

코드 6-9

클래스 메서드에서는 클래스 변수 사용만 가능

/6장/ClassMethods

```
01 class Program
02 {
03     public static int instanceVariable = 10;
04         └─ 추가해주었습니다.
05     static void Main(string[] args)
06     {
07         Console.WriteLine(instanceVariable);
08     }
09 }
```

04 오버로딩

- Math.Abs() 메서드를 사용할 때에 자동 완성 기능 확인

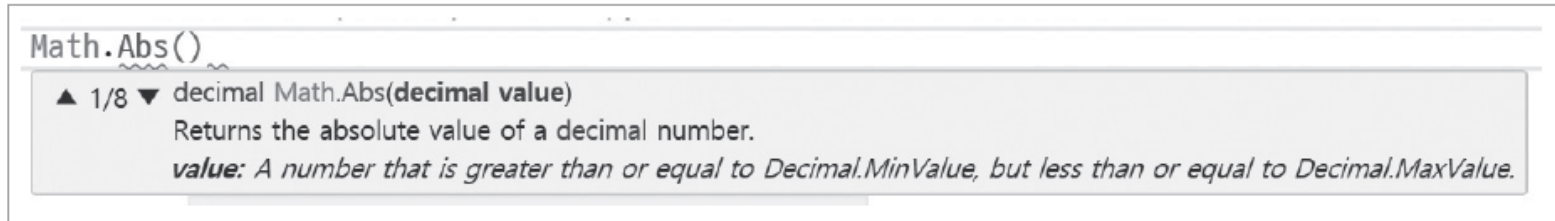


그림 6-6 Math.Abs() 메서드의 형태(1)

- 키보드의 위 아래 키를 눌러 움직이면 같은 Math.Abs() 메서드인데도 매개변수 형태가 다른 것들이 여러 개 있음

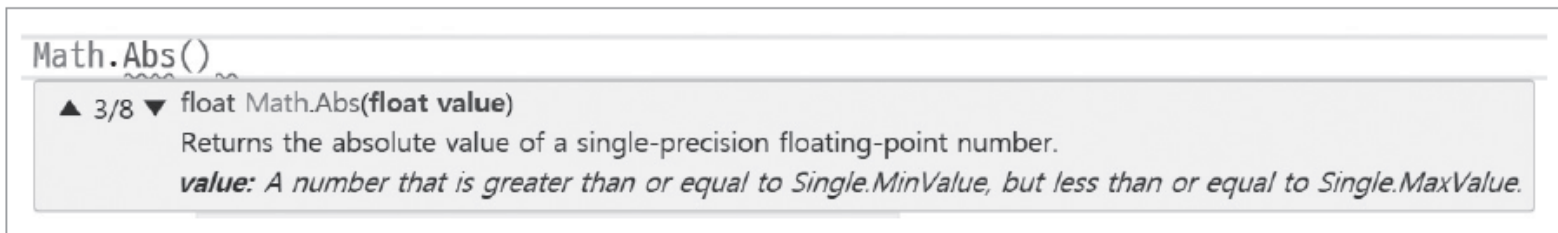


그림 6-7 Math.Abs() 메서드의 형태(2)

- 오버로딩(Overloading): 이름은 같고, 매개변수는 다른 메서드를 생성

04 오버로딩

■ 기본예제 6-5 메서드 오버로딩

/6장/Overloading

- [코드 6-10]은 이전에 만들었던 Abs() 메서드가 다양한 자료형에서 동작할 수 있게 오버로드함

코드 6-10 메서드 오버로딩

```
01 class Program
02 {
03     class MyMath
04     {
05         public static int Abs(int input)
06         {
07             if (input < 0) { return -input; }
08             else { return input; }
09         }
10
11         public static double Abs(double input)
12         {
13             if (input < 0) { return -input; }
14             else { return input; }
15         }
16
17         public static long Abs(long input)
18         {
19             if (input < 0) { return -input; }
20             else { return input; }
21         }
22     }
}
```

오버로딩했습니다.

04 오버로딩

■ 기본예제 6-5 메서드 오버로딩

/6장/Overloading

- [코드 6-10]은 이전에 만들었던 Abs() 메서드가 다양한 자료형에서 동작할 수 있게 오버로드함

```
23
24     static void Main(string[] args)
25     {
26         // int
27         Console.WriteLine(MyMath.Abs(52));
28         Console.WriteLine(MyMath.Abs(-273));
29
30         // double
31         Console.WriteLine(MyMath.Abs(52.273));
32         Console.WriteLine(MyMath.Abs(-32.103));
33
34         // long
35         Console.WriteLine(MyMath.Abs(21474836470));
36         Console.WriteLine(MyMath.Abs(-21474836470));
37     }
38 }
```

실행 결과

```
52
273
52.273
32.103
21474836470
21474836470
```

05 접근 제한자

- 가장 많이 사용되는 두 가지 접근 제한자

구분	접근 가능 범위	사용 목적
private	같은 클래스 내부	데이터 보호
public	어디서든	기능 공개

- 이 두 가지만 확실히 알아도 코딩하는 데 거의 문제가 없음

```
[접근 제한자] [자료형] [변수 이름]
[접근 제한자] [반환형] [메서드 이름]([매개변수])
{
    [메서드 코드]
}
```

05 접근 제한자

■ private 접근 제한자

- 접근 제한자를 입력하지 않으면 **자동으로 private 접근 제한자가 설정됨**
- 접근 제한자를 입력하지 않았으므로 Main() 메서드는 private 접근 제한자가 설정된 메서드

코드 6-12

Main() 메서드는 기본적으로 private 메서드

/6장/PrivateModifiers

```
01 static void Main(string[] args)
02 {
03
04 }
```

■ private 접근 제한자

- Program 클래스 내부에서 Program 클래스의 클래스 메서드를 확인

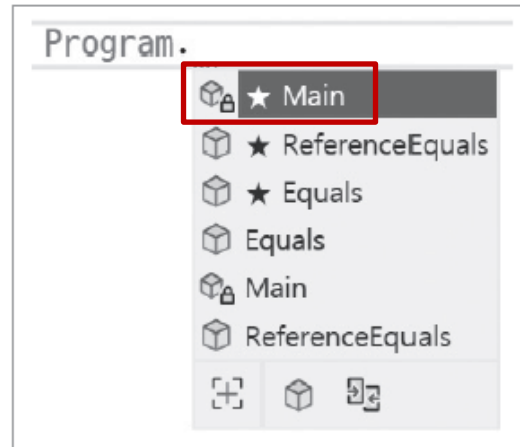


그림 6-11 자물쇠가 채워져 있는 Main() 메서드

- 자물쇠는 private 접근 제한자가 적용되어 있다는 의미
- **private 접근 제한자가 걸리면 자신의 클래스 내부에서만 해당 메서드를 사용할 수 있게 됨**

05 접근 제한자

■ private 접근 제한자

- 다른 클래스를 만들고 다른 클래스에서 Program 클래스의 Main() 메서드를 호출해보기

코드 6-13 외부 클래스에서의 접근

/6장/PrivateModifiers

```
01 class Test
02 {
03     public void TestMethod()
04     {
05         Program.Main(new string[] { "" });
06     }
07 }
08
09 class Program
10 {
11     static void Main(string[] args)
12     {
13
14     }
15 }
```

접근 제한자를 입력하지
않으면 **자동으로 private**
접근 제한자가 설정됨



■ private 접근 제한자

- 이 코드는 다음과 같은 오류가 발생
- Main() 메서드를 호출했지만 접근 제한자가 private이므로 다른 메서드에서는 접근할 수 없음



그림 6-12 접근 제한자로 인한 접근 불가 상태

05 접근 제한자

■ private 접근 제한자

- 다음 코드와 같이 자신의 클래스 내부에 있는 클래스의 메서드(내부 클래스 메서드) 또는 자신의 클래스 내부에 있는 메서드에서는 접근 가능

코드 6-14 내부 클래스에서의 접근

/6장/PrivateModifiers

```
01 class Program
02 {
03     class Test
04     {
05         public void TestMethod()
06         {
07             Program.Main(new string[] { "" });
08         }
09     }
10
11     public void TestMethod()
12     {
13         Program.Main(new string[] { "" });
14     }
15
16     static void Main(string[] args)
17     {
18
19     }
20 }
```

내부 클래스의 메서드에서 private 메서드를 접근했습니다.

자신의 클래스가 가진 메서드에서 private 메서드를 접근했습니다.

■ public 접근 제한자

- 다른 클래스에서 Main() 메서드를 호출하고 싶다면?
- public 접근 제한자를 붙여 공개 상태로 만들어줌

코드 6-15

public 접근 제한자를 붙인 Main() 메서드

/6장/PublicModifiers

```
01 class Test
02 {
03     public void TestMethod()
04     {
05         Program.Main(new string[] { "" });
06     }
07 }
08
09 class Program
10 {
11     public static void Main(string[] args)
12     {
13         접근 제한자를 추가했습니다.
14     }
15 }
```

05 접근 제한자

```
class Person
```



```
private int age; // 외부 접근 불가
```

참조 1개

```
public void SetAge(int value) // 외부에서 접근 가능
{
    if (value > 0)
        age = value;
}
```

참조 1개

```
public int GetAge() // 외부에서 접근 가능
{
    return age;
}
```

참조 0개

```
class Program
```

```
{
```

참조 0개

```
static void Main()
{
    Person p = new Person();

    // p.age = 10; 오류 (private이라 접근 불가)

    p.SetAge(25);
    Console.WriteLine(p.GetAge());
}
```

05 접근 제한자

접근 제한자	같은 클래스	다른 클래스 (상속 X)	다른 클래스 (상속 O)	다른 프로젝트
private	O	X	X	X
public	O	O	O	O
protected	O	X	O	X
internal	O	O	O	X

* 다른 클래스이지만 상속 관계에 있으면 가능

```
class Parent
{
    protected int x = 10;
}

class Child : Parent
{
    void Print()
    {
        Console.WriteLine(x); // 가능
    }
}
```

06 생성자

- 생성자(Constructor): 무언가를 생성할 때 자동으로 호출되는 메서드
- 인스턴스 생성자: 인스턴스를 생성할 때 자동으로 호출되는 메서드
- 인스턴스 생성자의 조건
 - 이름은 클래스 이름과 같아야 함
 - 접근 제한자는 public이어야 함
 - 반환과 관련된 선언을 하지 않아야 함

```
public [클래스 이름]([매개변수])  
{  
  
}
```

06 생성자

- 생성자에서는 일반적으로 인스턴스 변수를 초기화하는 일을 함

코드 6-16

기본적인 생성자의 모습

/6장/Constructors

```
01 class Product
02 {
03     public string name;
04     public int price;
05
06     public Product(string name, int price)
07     {
08         this.name = name;
09         this.price = price;
10     }
11 }
```

this 키워드는 클래스 자기 자신을 나타냅니다.
따라서 this.name은 자신의 name입니다.

this.price는 자신의 price입니다.

06 생성자

■ 기본예제 6-6 인스턴스 생성 개수 확인

/6장/ConstructorWithCounter

- 다음 코드는 클래스 변수 counter를 만들고 생성자에서 이를 증가시켜줌

코드 6-17 인스턴스 생성 개수 확인

```
01 class Program
02 {
03     class Product
04     {
05         public static int counter = 0;
06         public int id;
07         public string name;
08         public int price
09
10         public Product(string name, int price)
11         {
12             Product.counter = counter + 1;
13             this.id = counter;
14             this.name = name;
15             this.price = price;
16         }
17     }
18 }
```

06 생성자

■ 기본예제 6-6 인스턴스 생성 개수 확인

/6장/ConstructorWithCounter

- 다음 코드는 클래스 변수 counter를 만들고 생성자에서 이를 증가시켜줌

```
19     static void Main(string[] args)
20     {
21         Product productA = new Product("감자", 2000);
22         Product productB = new Product("고구마", 3000);
23
24         Console.WriteLine(productA.id + ":" + productA.name);
25         Console.WriteLine(productB.id + ":" + productB.name);
26         Console.WriteLine(Product.counter + "개 생성되었습니다.");
27     }
28 }
```

실행 결과

1:감자
2:고구마
2개 생성되었습니다.

■ 정적 생성자

- 정적 생성자: 정적 요소를 초기화할 때에 사용되는 생성자

코드 6-19

정적 생성자

/6장/Constructors

```
01 class Sample
02 {
03     public static int value;
04
05     static Sample()
06     {
07         value = 10;
08         Console.WriteLine("정적 생성자 호출");
09     }
10 }
```

- 정적 생성자를 만들 때는 다음과 같은 제한이 있음
 - 접근 제한자를 사용하지 못함
 - 매개변수를 사용하지 못함

■ 정적 생성자

- 정적 생성자는 정적 요소를 사용할 때,
또는 인스턴스를 생성하는 초기 시점에 한 번만 호출됨
- 두 코드 모두 해당 클래스와 관련된 요소를 처음 사용하는 시점에 자동적으로 호출됨

코드 6-20(1) 정적 생성자의 호출 시점

/6장/Constructors

```
01 static void Main(string[] args)
02 {
03     Console.WriteLine("첫 번째 위치");
04     Console.WriteLine(Sample.value); —— 이때 한 번만 호출됩니다.
05     Console.WriteLine("두 번째 위치");
06     Sample sample = new Sample();
07     Console.WriteLine("세 번째 위치");
08 }
```

코드 6-20(2) 정적 생성자의 호출 시점

/6장/Constructors

```
01 static void Main(string[] args)
02 {
03     Console.WriteLine("첫 번째 위치");
04     Sample sample = new Sample(); —— 이때 한 번만 호출됩니다.
05     Console.WriteLine("두 번째 위치");
06     Console.WriteLine(Sample.value);
07     Console.WriteLine("세 번째 위치");
08 }
```

■ 소멸자

- 소멸자(Destructor): 인스턴스가 소멸될 때에 호출됨
- C#은 가비지 컬렉터가 알아서 최적화해서 객체를 소멸시킴

코드 6-21 소멸자

```
01 class Product()  
02 {  
03     public string name;  
04     public int price;  
05  
06     public Product(string name, int price)  
07     {  
08         this.name = name;  
09         this.price = price;  
10     }  
11  
12     ~Product()  
13     {  
14         Console.WriteLine(this.name + "의 소멸자 호출");  
15     }  
16 }
```

■ 상수

- 현재 만든 인스턴스 변수와 클래스 변수는 다음과 같이 생김

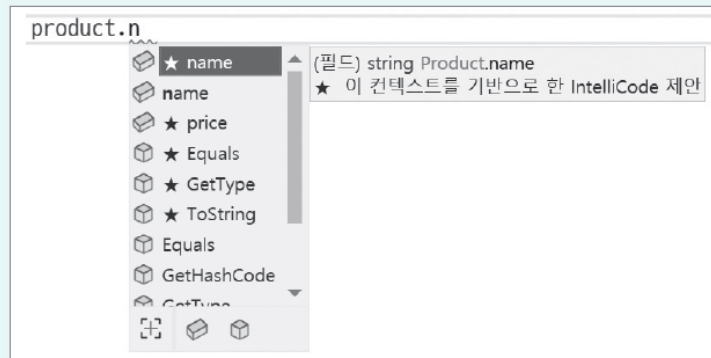


그림 6-13 일반적인 인스턴스 변수

- Math 클래스의 PI 속성은 다음과 같이 생김

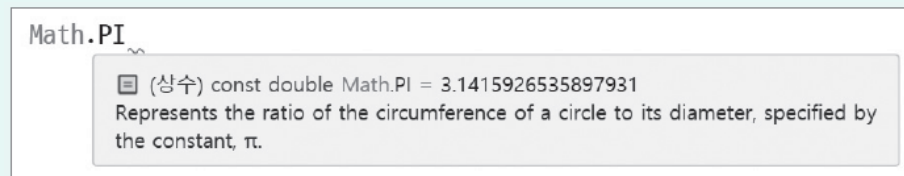


그림 6-14 상수

■ 상수

- Math 클래스의 PI 속성은 상수(Constant)이기 때문에 변수의 값을 변경할 수 없음

코드 6-22

상수 변경

/6장/Constants

```
01 static void Main(string[] args)
02 {
03     Math.PI = 20;
04
05     int r = 10;
06     Console.WriteLine("원의 둘레: " + (2 * Math.PI * r));
07     Console.WriteLine("원의 넓이: " + (Math.PI * r * r));
08 }
```

- 상수로 선언되어 있기 때문에 이렇게 값을 변경하는 데 오류가 발생

Math.PI = 20;

 class System.Math

Provides constants and static methods for trigonometric, logarithmic, and other common mathematical functions.

CS0131: 할당식의 왼쪽은 변수, 속성 또는 인덱서여야 합니다.

그림 6-15 상수를 변경할 때의 오류

■ 상수

- 값을 변경할 필요가 없거나 값을 변경하는 것이 위험할 때에 상수를 사용
- 변수 또는 속성 앞에 `const` 키워드를 붙여주면 됨

코드 6-23

상수 생성

/6장/Constants

```
01 class MyMath
02 {
03     public const double PI = 3.141592;
04 }
```

- `const` 키워드는 일반적인 메서드 내부에도 사용할 수 있음

코드 6-24

메서드 내부에서 상수 사용

/6장/Constants

```
01 static void Main(string[] args)
02 {
03     const int value = 10;
04
05     Console.WriteLine(value);
06 }
```


07 속성

- 학교에서 팀 프로젝트 과제를 하다가 다음과 같은 Box 클래스를 만들었다고 함
- Box 클래스는 너비width와 높이height를 변수로 갖고 있음
- 사각형의 넓이를 구하는 Area() 메서드도 있음

코드 6-26

Box 클래스

/6장/Boxes

```
01 class Box
02 {
03     public int width;
04     public int height;
05
06     public Box(int width, int height)
07     {
08         this.width = width;
09         this.height = height;
10     }
11
12     public int Area()
13     {
14         return this.width * this.height;
15     }
16 }
```

07 속성

- Area() 메서드를 호출했더니 음수가 튀어나옴
- 다른 팀원이 다음과 같이 너비를 음수로 입력해버린 것을 발견함

```
Box box = new Box(10, 10);  
box.width = -10;
```

- 클래스를 만들 때에 이러한 일을 원천적으로 봉쇄해야 함

■ 캡슐화

- 다른 사람이 width와 height를 못 건드리게 다음과 같이 코드를 수정할 수 있음
- 접근 제한자를 private으로 변경하고 생성자에서만 값을 변경하도록 함

코드 6-27 변수 width와 height를 건드리지 못하게 수정

/6장/Boxes

```
01 class Box
02 {
03     private int width;
04     private int height;
05
06     public Box(int width, int height)
07     {
08         if (width > 0 || height > 0)
09         {
10             this.width = width;
11             this.height = height;
12         }
13         else
14         {
15             Console.WriteLine("너비와 높이는 자연수로 초기화해주세요!");
16         }
17     }
```

■ 캡슐화

```
18  
19     public int Area()  
20     {  
21         return this.width * this.height;  
22     }  
23 }
```

- 이제 한 번 생성한 width와 height를 바꿀 수 없고
width와 height에 어떤 값이 들어있는지 확인조차 할 수 없음

■ 게터와 세터

- public으로 변수를 선언해버리면 외부에서 마음대로 값을 설정해버릴 수 있고, 음수를 넣을 수 있게 됨
- private으로 변수를 선언하면 외부에서 아예 값을 건드릴 수 없음
- 이럴 때는 변수를 바로 건드릴 수는 없지만 변수를 변경하는 메서드를 만들고, 메서드를 호출해 변경하면 됨

코드 6-28

게터와 세터

/6장/Boxes

```
01 class Box
02 {
03     // 변수
04     private int width;
05     private int height;
06
```

■ 게터와 세터

```
07     // 생성자
08     public Box(int width, int height)
09     {
10         if (width > 0 || height > 0)
11         {
12             this.width = width;
13             this.height = height;
14         }
15         else { Console.WriteLine("너비와 높이는 자연수로 초기화해주세요!"); }
16     }
17
18     // 인스턴스 메서드
19     public int Area() { return this.width * this.height; }
20
```

■ 게터와 세터

```
21 // 게터(Getter)
22 public int GetWidth() { return width; }
23 public int GetHeight() { return height; }
24
25 // 세터(Setter)
26 public void SetWidth(int width)
27 {
28     if (width > 0) { this.width = width; }
29     else { Console.WriteLine("너비는 자연수를 입력해주세요"); }
30 }
31
32 public void SetHeight(int height)
33 {
34     if (height > 0) { this.height = height; }
35     else { Console.WriteLine("높이는 자연수를 입력해주세요"); }
36 }
37 }
```

- 게터: 값을 추출할 때에 사용한 GetWidth()와 GetHeight() 메서드
- 세터: 값을 넣을 때에 사용한 SetWidth()와 SetHeight() 메서드
- 게터와 세터를 사용하면 안전하게 값을 넣고 뺄 수 있음

07 속성

■ 일반적인 속성 생성 방법

- 속성: 게터와 세터를 손쉽게 만들 수 있는 방법
- 속성을 만드는 방법은 다음과 같음

```
private int [변수 이름];  
public int [속성 이름]  
{  
    get { return [변수 이름]; }  
    set { [변수 이름] = value; }  
}
```

- 이렇게 만들어진 속성은 다음과 같이 사용

```
[인스턴스 이름].[속성 이름]          // 게터 호출  
[인스턴스 이름].[속성 이름] = [값]    // 세터 호출
```


■ 기본예제 6-7 일반적인 속성

/6장/BoxProperty

- 다음 코드는 이전의 게터와 세터를 속성으로 변환한 코드

코드 6-29 일반적인 속성

```
01 class Program
02 {
03     class Box
04     {
05         // 변수와 속성
06         private int width;
07         public int Width —— 속성 Width를 생성했습니다.
08         {
09             get { return width; }
10             set
11             {
12                 if (value > 0) { width = value; }
13                 else { Console.WriteLine("너비는 자연수를 입력해주세요."); }
14             }
15         }
16
17         private int height;
18         public int Height —— 속성 Height를 생성했습니다.
```

■ 기본예제 6-7 일반적인 속성

/6장/BoxProperty

- 다음 코드는 이전의 게터와 세터를 속성으로 변환한 코드

```
19         {
20             get { return height; }
21             set
22             {
23                 if (value > 0) { height = value; }
24                 else { Console.WriteLine("높이는 자연수를 입력해주세요."); }
25             }
26         }
27
28         // 생성자
29         public Box(int width, int height)
30         {
31             Width = width;
32             Height = height;
33         }
34
35         // 인스턴스 메서드
36         public int Area() { return this.width * this.height; }
37     }
38
39     static void Main(string[] args)
```

07 속성

■ 기본예제 6-7 일반적인 속성

/6장/BoxProperty

- 이렇게 만들어진 속성은 사용할 때에 일반 변수와 다르게 다음과 같이 표시됨
- 발할 때 자동 완성에  기호가 뜬다면, 속성을 의미

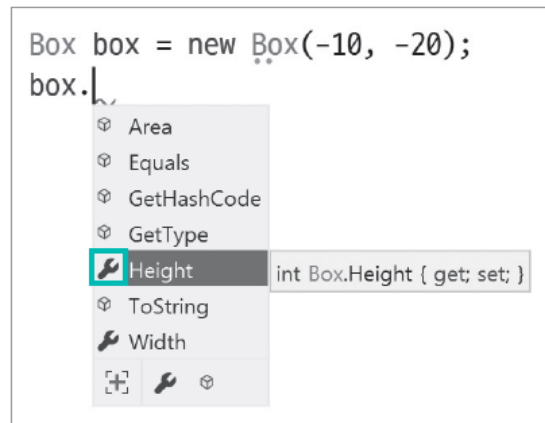


그림 6-17 속성 기호

07 속성

■ 간단한 속성 생성 방법

- 프레임워크는 변수가 아니라 속성을 강제하는 경우가 있는데, 안전 측면에서는 좋은 확인이지만 속성을 10개, 20개 입력하다 보면 엄청나게 귀찮을 것
- 간단하게 속성을 생성하는 방법이 있음

```
public int Width { get; set; }  
public int Height { get; set; }
```

■ 속성 코드 조각

- 속성은 많은 개발자들이 외우기 힘들어하는 내용이므로 코드 조각이 마련되어 있음
- 자동 완성 기능이 켜진 상태라면 그냥, 아니라면 [Ctrl] + [Space] 단축키를 입력하고 prop라고 입력

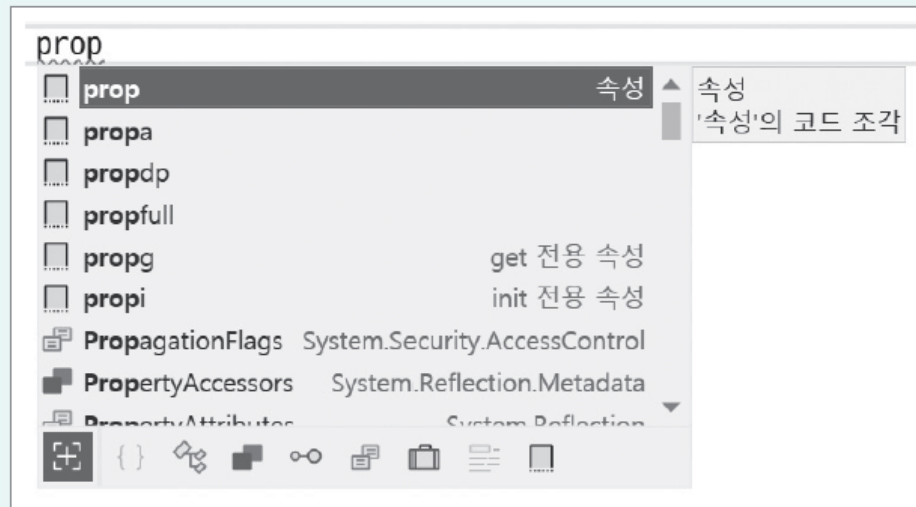


그림 6-18 prop 코드 조각

여기서 잠깐!

■ 속성 코드 조각

- prop을 잡고 [Tab] 키를 두 번 누르면 다음과 같은 코드가 자동으로 생성됨

```
public int MyProperty { get; set; }
```

- propfull을 잡고 [Tab] 키를 두 번 누르면 다음과 같은 코드가 자동으로 생성됨

```
private int myVar;

public int MyProperty
{
    get { return myVar; }
    set { myVar = value; }
}
```

08 값 복사와 참조 복사

- C#의 모든 자료형은 값Value 또는 참조Reference 두 가지로 나뉨
 - 값: int, float 등 기본 자료형Primitive Type
 - 참조: 클래스로 만들어진 인스턴스(객체)
- 기본 자료형을 선언하는 경우
 - int 자료형을 선언하면 메모리에 곧바로 4바이트의 영역을 잡고 값이 들어감

```
static void Main(string[] args)
{
    int a = 10;
}
```

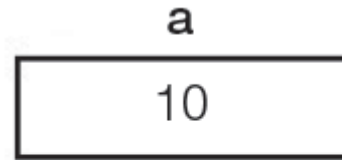


그림 6-19 기본 자료형 선언

08 값 복사와 참조 복사

- 객체를 선언할 때, `new Random()`으로 인스턴스를 생성하면 인스턴스는 힙(Heap)이라는 영역에 위치를 잡고 주소값이 변수 `a`에 들어감

```
static void Main(string[] args)
{
    Random a = new Random();
}
```

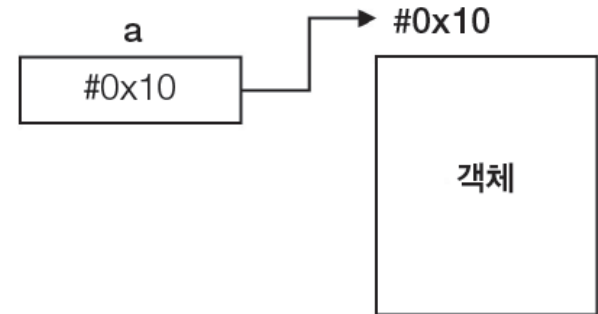


그림 6-20 객체 자료형 선언

08 값 복사와 참조 복사

■ 값 복사의 경우

코드 6-30

값 복사의 예

```
01 class Program
02 {
03     static void Change(int input)
04     {
05         input = 20;
06     }
07
08     static void Main(string[] args)
09     {
10         int a = 10;
11         Change(a);
12         Console.WriteLine(a);
13     }
14 }
```

실행 결과

10



그림 6-21 값 복사의 과정

08 값 복사와 참조 복사

■ 값 복사의 경우

- input을 20으로 바꾸면 [그림 6-22]와 같이 됨
- 변수 a와 변수 input은 독립적인 변수이므로 서로를 변경해도 영향을 주지 않음



그림 6-22 값 복사된 변수를 변경

08 값 복사와 참조 복사

■ 값 복사의 경우

코드 6-31 간단한 값 복사의 예

```
01 static void Main(string[] args)
02 {
03     int a = 10;
04     int input = a;
05     input = 20;
06     Console.WriteLine(a);
07 }
```

- 결과는 마찬가지로 10이 나옴

08 값 복사와 참조 복사

■ 참조 복사의 경우

코드 6-32 참조 복사의 예

```
01 class Program
02 {
03     class Test
04     {
05         public int value = 10;
06     }
07
08     static void Change(Test input)
09     {
10         input.value = 20;
11     }
12
13     static void Main(string[] args)
14     {
15         Test test = new Test();
16         test.value = 10;
17         Change(test);
18
19         Console.WriteLine(test.value);
20     }
21 }
```

실행 결과

20

08 값 복사와 참조 복사

■ 참조 복사의 경우

- 객체는 변수에 값이 아니라 참조(객체의 메모리 위치를 나타내는 주소)가 들어감
- 메서드를 호출할 때도 참조가 복사됨

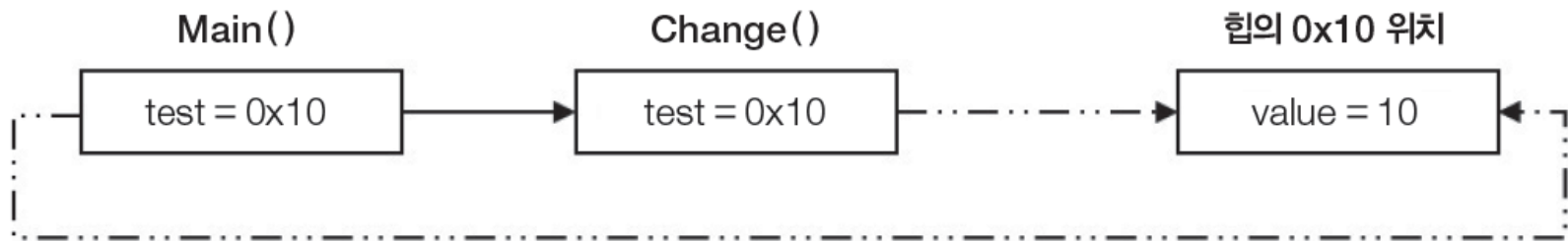


그림 6-23 참조 복사의 과정

08 값 복사와 참조 복사

■ 참조 복사의 경우

- Change() 메서드 내부에서 test.value를 변경하면 힙 공간에 있는 value가 변경됨

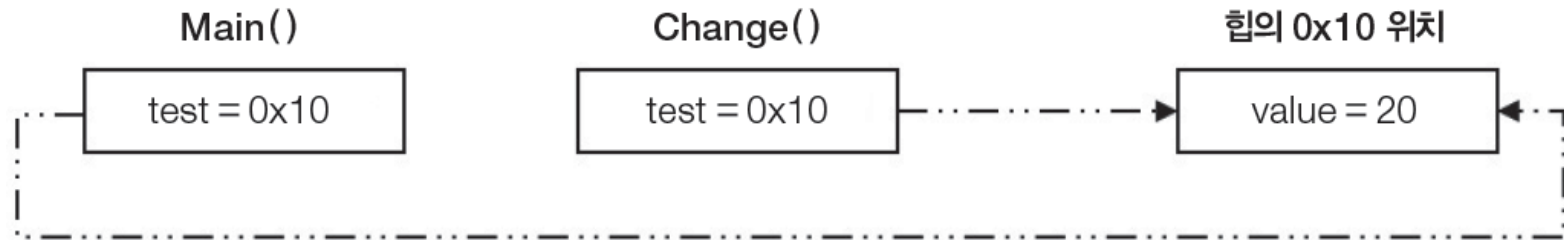


그림 6-24 참조 복사된 객체를 변경

08 값 복사와 참조 복사

■ 참조 복사의 경우

- 마찬가지로 20이 출력됨

코드 6-33

간단한 참조 복사 예

```
01 static void Main(string[] args)
02 {
03     Test testA = new Test();
04     Test testB = testA;
05     testA.value = 10;
06     testB.value = 20;
07     Console.WriteLine(testA.value);
08 }
```

- 기본 자료형에는 값이 들어있고 객체에는 참조가 들어있다고만 이해하면 됨

09 함께하는 응용예제

■ 응용예제 6-1 재귀 메서드

/6장/Recursion

- 어떠한 메서드 내부에서 자기 자신을 호출하게 되면, 이 메서드를 재귀 메서드라고 부름

코드 6-34 재귀 메서드

```
01 class Program
02 {
03     static void Main(string[] args)
04     {
05         Main(new string[0]); —— 자기 자신을 호출합니다.
06     }
07 }
```

- 위의 메서드를 호출하게 되면 자기 자신을 계속 호출하게 되어 종료되지 않음
- 재귀 메서드에는 항상 자기 자신을 탈출할 수 있는 종료 조건이 포함되어야 함

09 함께하는 응용예제

■ 응용예제 6-1 재귀 메서드

/6장/Recursion

- 피보나치 수를 만들어보며 재귀 메서드를 만들어보기

코드 6-35 재귀 메서드를 사용한 피보나치 인스턴스 메서드

```
01 class Fibonacci
02 {
03     public long Get(int i)
04     {
05         if (i <= 0) { return 0; }
06         if (i == 1) { return 1; }
07         return Get(i - 2) + Get(i - 1);
08     }
09 }
10
11 class Program
12 {
13     static void Main(string[] args)
14     {
15         Fibonacci fibo = new Fibonacci();
16         Console.WriteLine(fibo.Get(1));
17         Console.WriteLine(fibo.Get(2));
18         Console.WriteLine(fibo.Get(3));
19         Console.WriteLine(fibo.Get(4));
20         Console.WriteLine(fibo.Get(5));
21     }
22 }
```

피보나치 수는 기하급수적으로 커지므로 long 자료형을 사용하게 했습니다.

종료 조건입니다.

$$F_n := \begin{cases} 0 & \text{if } n = 0; \\ 1 & \text{if } n = 1; \\ F_{n-1} + F_{n-2} & \text{if } n = 1. \end{cases}$$

그림 6-25 피보나치 수열(그림 출처: 위키백과)

실행 결과

1
1
2
3
5

09 함께하는 응용예제

■ 응용예제 6-1 재귀 메서드

/6장/Recursion

- Get() 메서드를 클래스 메서드로 바꿔주면 조금 더 그럴듯한 코드가 됨

코드 6-36 재귀 메서드를 사용한 피보나치 클래스 메서드

```
01 class Fibonacci
02 {
03     public static long Get(int i)
04     {
05         if (i <= 0) { return 0; }
06         if (i == 1) { return 1; }
07         return Get(i - 2) + Get(i - 1);
08     }
09 }
10
11 class Program
12 {
13     static void Main(string[] args)
14     {
15         Console.WriteLine(Fibonacci.Get(1));
16         Console.WriteLine(Fibonacci.Get(2));
17         Console.WriteLine(Fibonacci.Get(3));
18         Console.WriteLine(Fibonacci.Get(4));
19         Console.WriteLine(Fibonacci.Get(5));
20     }
21 }
```

클래스 메서드로 선언했습니다.

09 함께하는 응용예제

■ 응용예제 6-2 메모화

/6장/Memorize

- 방금 전에 살펴보았던 재귀 메서드와 재귀 인덱서에는 큰 문제가 있음

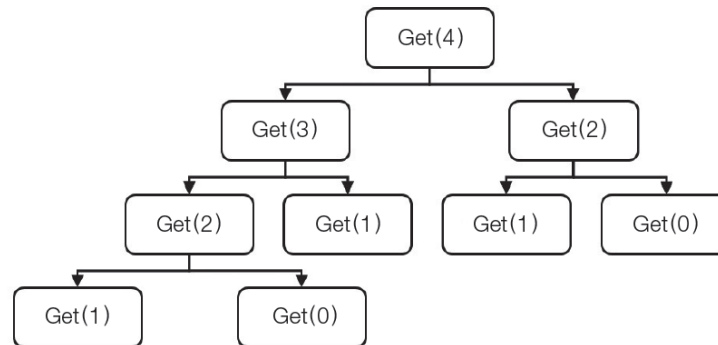


그림 6-26 피보나치 4를 구하는 과정

- 같은 피보나치 수를 계속해서 구하게 되어 프로그램의 속도가 굉장히 느려짐
- 이런 문제를 해결할 때는 메모화라는 기술을 사용
 - 메모화: 한 번 계산했던 값을 저장해두는 기술

09 함께하는 응용예제

■ 응용예제 6-2 메모화

/6장/Memorize

코드 6-37 메모화

```
01 class Fibonacci
02 {
03     private static Dictionary<int, long> memo = new Dictionary<int, long>();
04     public static long Get(int i)
05     {
06         // 기본 값
07         if (i < 0) { return 0; }
08         if (i == 1) { return 1; }
09
10         // 이미 계산했던 값인지 확인합니다.
11         if (memo.ContainsKey(i))
12         {
13             return memo[i];
14         }
15         else
16         {
17             // 계산하지 않았다면 계산합니다.
```

계산한 피보나치 수를 저장하는 Dictionary 객체를 만듭니다.

실행 결과

102334155

3736710778780434371

09 함께하는 응용예제

■ 응용예제 6-2 메모화

/6장/Memorize

```
18         long value = Get(i - 2) + Get(i - 1);
19         memo[i] = value;
20         return value;
21     }
22 }
23 }
24
25 class Program
26 {
27     static void Main(string[] args)
28     {
29         Console.WriteLine(Fibonacci.Get(40));
30         Console.WriteLine(Fibonacci.Get(100));
31     }
32 }
```

실행 결과

102334155

3736710778780434371

- 반복문으로 피보나치 수열을 구하는 방법도 생각해보기

10 윈도 폼: 윈도 폼에서 메서드 활용하기

■ 이벤트 정적 연결

- 버튼을 클릭했을 때에 무슨 일이 생기게 만들어보기
- 일단 도구 상자의 버튼(Button 클래스), 텍스트 박스(TextBox 클래스), 레이블(Label 클래스)로 다음과 같이 화면을 구성



그림 6-27 화면 디자인

10 윈도 폼: 윈도 폼에서 메서드 활용하기

■ 이벤트 정적 연결

코드 6-38 Form1.Designer.cs 파일에 자동 생성된 요소

```
01 private Button button1;  
02 private TextBox textBox1;  
03 private Label label1;
```

- 디자인 화면에서 생성된 버튼을 클릭하고 속성 화면의 번개 모양 아이콘(이벤트)을 누르면 이벤트가 다양하게 나옴

10 윈도 폼: 윈도 폼에서 메서드 활용하기

■ 이벤트 정적 연결

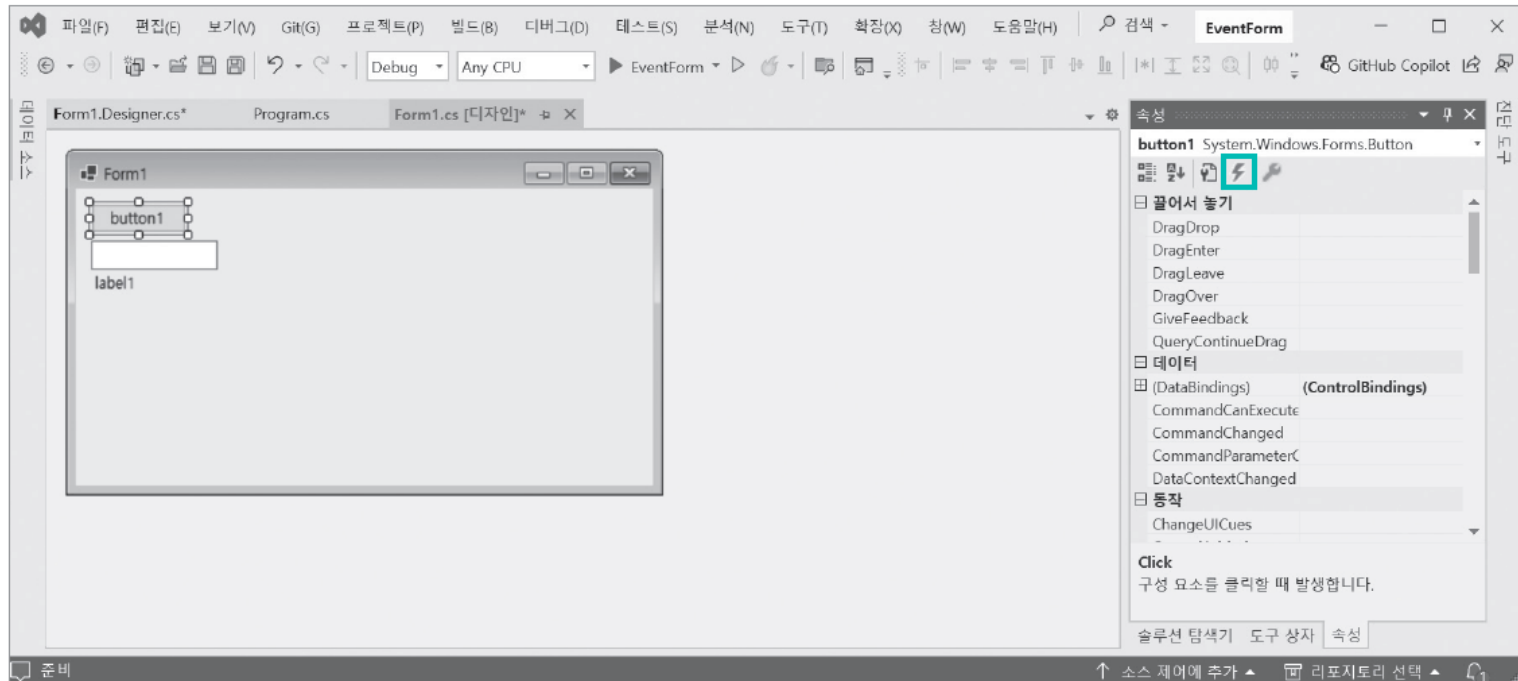


그림 6-28 버튼에 적용할 수 있는 이벤트

- 속성 화면의 Click 글자 자체 또는 옆에 있는 빈 공간을 잡고 더블클릭
- 더블클릭하면 다음과 같이 코드 화면으로 넘어가게 됨

10 윈도 폼: 윈도 폼에서 메서드 활용하기

■ 이벤트 정적 연결

코드 6-39 자동으로 생성된 이벤트 메서드

```
01 public partial class Form1 : Form
02 {
03     public Form1()
04     {
05         InitializeComponent();
06     }
07
08     private void button1_Click(object? sender, EventArgs e)
09     {
10
11     }
12 }
```

자동으로 생성된 이벤트 메서드
입니다. 버전에 따라서 매개변
수가 일부 다를 수 있는데 자동
으로 생성된 메서드 형태 그대로
사용하면 됩니다.

- 다시 디자인 화면으로 와서 속성 화면을 보면 Click 부분에 자동 생성된 button1_Click() 메서드가 연결된 것을 확인할 수 있음

10 윈도 폼: 윈도 폼에서 메서드 활용하기

■ 이벤트 정적 연결

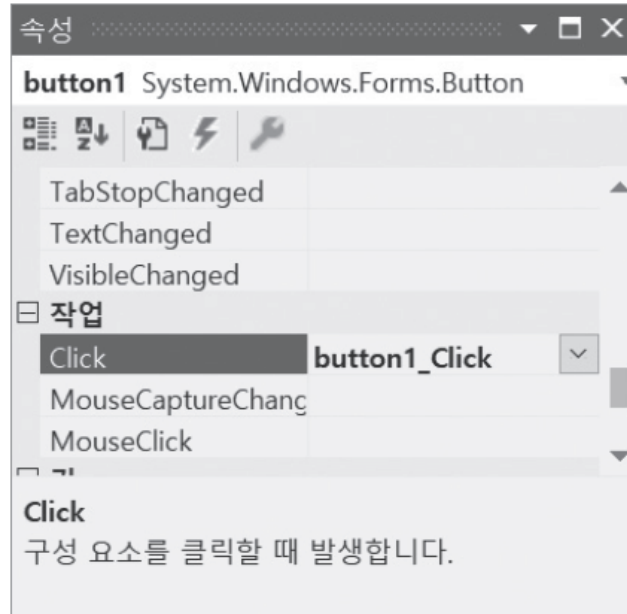


그림 6-29 연결된 이벤트

- 이렇게 이벤트가 연결되면 버튼을 클릭할 때 `button1_Click()` 메서드를 실행하게 된 것
- `button1_Click()` 메서드에 코드를 작성하면 이는 버튼을 클릭할 때 실행됨

10 윈도 폼: 윈도 폼에서 메서드 활용하기

■ 이벤트 정적 연결

코드 6-40 button1_Click() 메서드

```
01 private void button1_Click(object? sender, EventArgs e)
02 {
03     textBox1.Text += "+";
04     label1.Text += "+";
05 }
```

- 프로젝트를 실행하고 버튼을 클릭하면 다음과 같이 됨

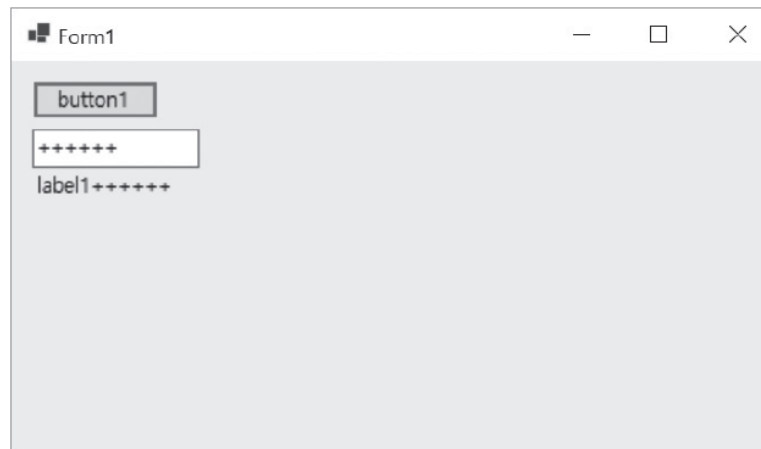


그림 6-30 버튼을 여러 번 클릭했을 때의 화면

10 윈도 폼: 윈도 폼에서 메서드 활용하기

■ 이벤트 동적 연결

- 코드에서 이벤트를 연결하는 법을 알아보기
- Form1.Designer.cs 파일을 열면 다음과 같이 자동 생성된 코드가 있음

코드 6-41

Form1.Designer.cs 파일에 자동 생성된 코드

```
01 button1.Location = new Point(12, 12);
02 button1.Name = "button1";
03 button1.Size = new Size(75, 23);
04 button1.TabIndex = 0;
05 button1.Text = "button1";
06 button1.UseVisualStyleBackColor = true;
07 button1.Click += button1_Click;————— 여기가 바로 이벤트를 연결하는 부분입니다.
```

10 윈도 폼: 윈도 폼에서 메서드 활용하기

■ 이벤트 동적 연결

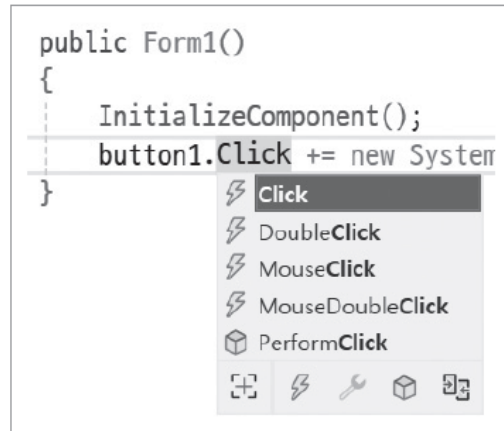


그림 6-31 번개 모양의 이벤트

- 비주얼 스튜디오는 이벤트 메서드를 자동으로 작성해줌
- 이벤트 뒤에 += 기호를 입력하면 다음과 같은 설명이 뜸

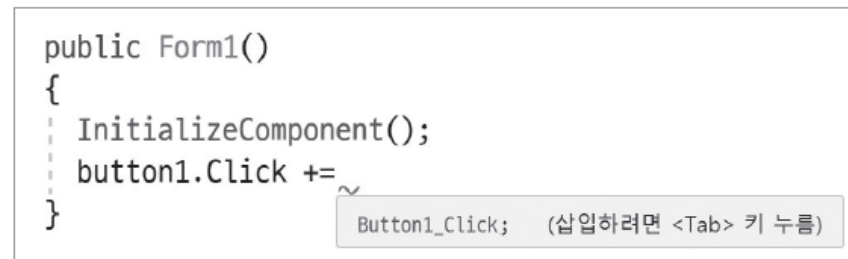


그림 6-32 삽입하려면 `Tab` 키를 누름

10 윈도 폼: 윈도 폼에서 메서드 활용하기

■ 이벤트 동적 연결

코드 6-42 자동 생성된 이벤트 메서드

```
01 public partial class Form1 : Form
02 {
03     public Form1()
04     {
05         InitializeComponent();
06
07         button1.Click += Button1_Click;
08     }
09
10     private void Button1_Click(object? sender, EventArgs e)
11     {
12         throw new NotImplementedException();
13     }
14
15     private void button1_Click(object? sender, EventArgs e)
16     {
17         textBox1.Text += "+";
18         label1.Text += "+";
19     }
20 }
```

구현되지 않았으니 구현해달라고 강제로 오류를 띄우는 코드입니다. 지금 버튼을 클릭하면 강제로 예외가 발생합니다. 이와 관련된 내용은 10장에서 다루지만 지금 기억해두면 그때 도움이 될 것입니다.

10 윈도 폼: 윈도 폼에서 메서드 활용하기

■ 이벤트 메서드의 매개변수

- 이벤트 메서드는 다음과 같은 형태

```
private void Button1_Click(object? sender, EventArgs e)
{

}

```

- 대부분의 GUI 프레임워크는 대부분 두개의 매개변수를 갖음
 - Sender 객체
 - 이벤트 정보 객체

10 윈도 폼: 윈도 폼에서 메서드 활용하기

■ 이벤트 메서드의 매개변수

■ Sender 객체

- sender는 이벤트를 발생시킨 자기 자신을 나타냄
- 만약 버튼을 클릭해서 해당 이벤트 메서드가 실행되었다면 sender는 버튼을 의미함
- 다음과 같은 코드를 사용하면 버튼을 클릭한 자기 자신의 Text 속성을 변경할 수 있음

코드 6-43 sender 객체 활용

```
01 private void Button1_Click(object? sender, EventArgs e)
02 {
03     Button self = (Button)sender;
04     self.Text = "저를 클릭했습니다!";
05 }
```


10 윈도 폼: 윈도 폼에서 메서드 활용하기

■ 이벤트 메서드의 매개변수

■ 이벤트 정보 객체

- 이벤트와 관련된 추가 정보를 알려주는 객체
- 그런데 다음과 같이 무슨 속성과 메서드가 있는지 보려고 하면 아무것도 없음

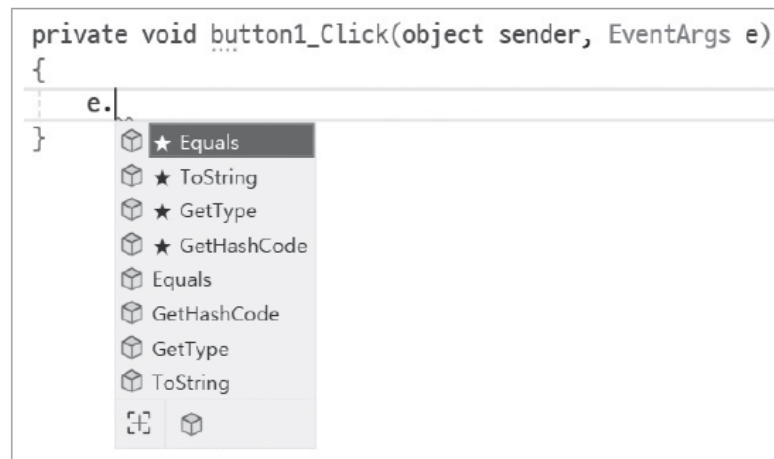


그림 6-33 이벤트 정보 객체의 멤버

- 버튼 클릭이라는 이벤트는 딱히 추가적으로 살펴볼 이벤트가 없다는 것이므로 다른 이벤트를 살펴봐야 함

10 윈도 폼: 윈도 폼에서 메서드 활용하기

■ 이벤트 메서드의 매개변수

■ 이벤트 정보 객체

코드 6-44 FormClosed 이벤트

```
01 public partial class Form1 : Form
02 {
03     public Form1()
04     {
05         InitializeComponent();
06         button1.Click += Button1_Click;
07         FormClosed += Form1_FormClosed;
08     }
09
10
11     private void Form1_FormClosed(object sender, FormClosedEventArgs e)
12     {
13         throw new NotImplementedException();
14     }
15
16     private void Button1_Click(object? sender, EventArgs e) { /* 생략 */ }
17     private void button1_Click(object? sender, EventArgs e) { /* 생략 */ }
18 }
```

자기 자신(Form1 클래스의 인스턴스)의 FormClosed 이벤트입니다.
이는 부모에서 상속받은 이벤트인데요. 상속은 다음 장에서 알아보겠습니다.

이벤트 정보 객체의 형식이 이전과 다릅니다.

10 윈도 폼: 윈도 폼에서 메서드 활용하기

■ 이벤트 메서드의 매개변수

■ 이벤트 정보 객체

- 이제 추가 정보가 조금 나옴

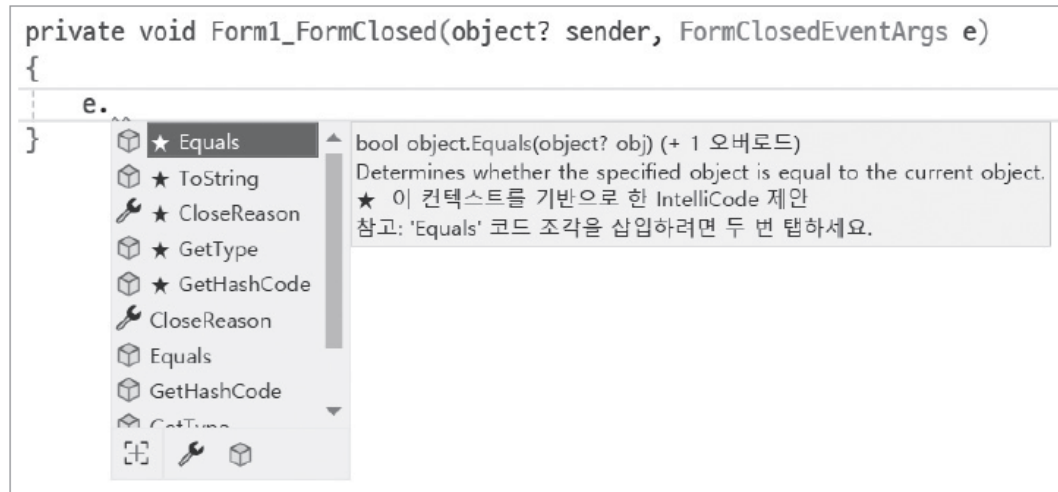


그림 6-34 이벤트 정보 객체의 다른 멤버

- 이벤트 정보 객체: 특정한 이벤트에서 추가적인 정보를 얻을 수 있음

10 윈도 폼: 윈도 폼에서 메서드 활용하기

■ 백그라운드 요소의 이벤트

- 윈도 폼은 눈에 보이지 않는 요소가 있음
- 도구 상자의 Components(구성 요소)를 살펴보고 Timer를 화면에 배치

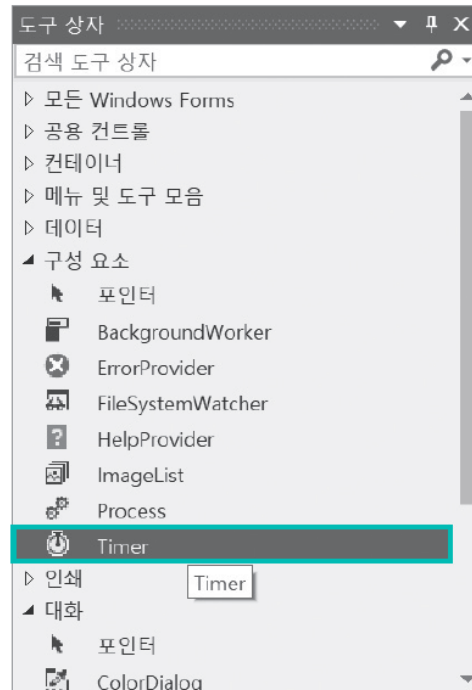


그림 6-35 도구 상자의 구성 요소

10 윈도 폼: 윈도 폼에서 메서드 활용하기

■ 백그라운드 요소의 이벤트

- 여기에 있는 요소들은 화면에 드래그해도 화면에 나오지 않고 [그림 6-36]처럼 별도의 공간에 들어감

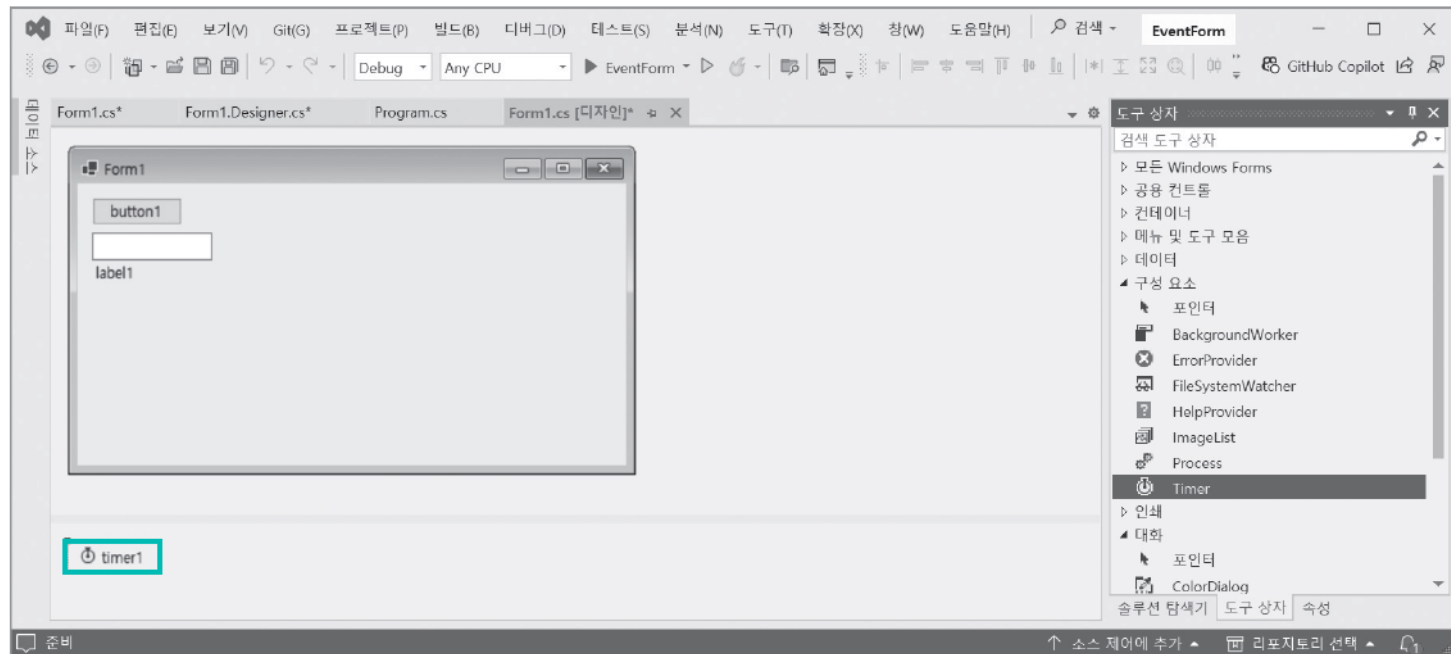


그림 6-36 타이머(Timer 클래스)

10 윈도 폼: 윈도 폼에서 메서드 활용하기

■ 백그라운드 요소의 이벤트

- 버튼을 누르면 버튼의 Enabled 속성을 true로 바꿔서 타이머를 작동시키고, 프로그램을 실행한 지 몇 초나 지났는지 확인하는 프로그램
- Enabled 속성을 true로 바꾸고, Interval 속성을 1000(밀리초)으로 입력해서 1초 간격으로 실행하는 타이머로 변경
- 이벤트에서 Tick을 잡고 오른쪽의 빈 공간을 더블클릭

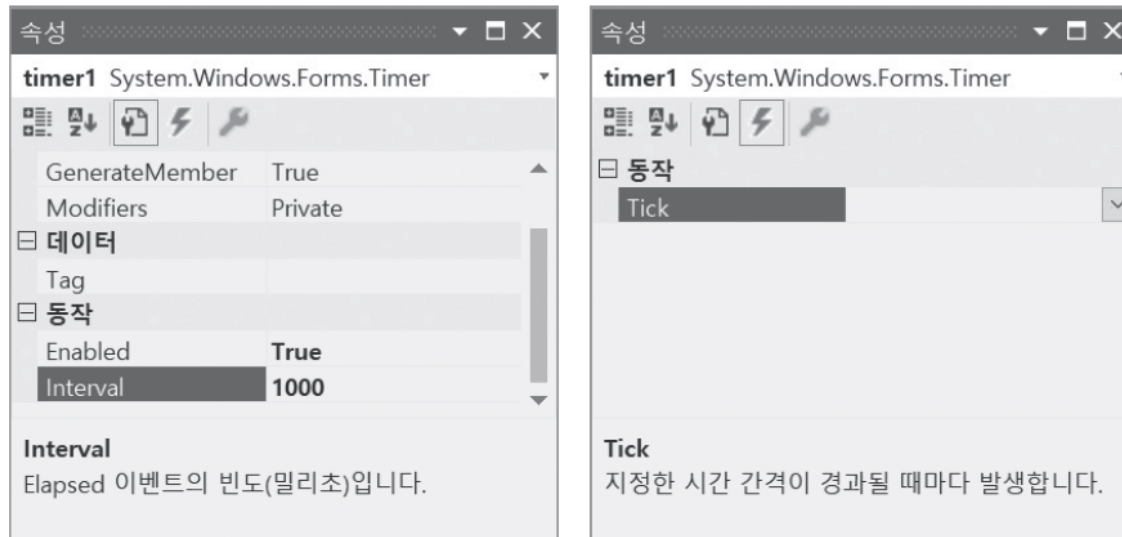


그림 6-37 타이머 객체의 속성 지정

10 윈도 폼: 윈도 폼에서 메서드 활용하기

■ 백그라운드 요소의 이벤트

- 이벤트 메서드가 다음과 같이 자동 생성될 것

코드 6-45 Tick 이벤트의 이벤트 메서드

```
01 private void timer1_Tick(object? sender, EventArgs e)
02 {
03
04 }
```

10 윈도 폼: 윈도 폼에서 메서드 활용하기

■ 백그라운드 요소의 이벤트

- 내부에는 다음과 같은 코드를 입력해줌

코드 6-46 시간 경과 확인 코드

```
01 private int elapsedTime = 0;
02 private void timer1_Tick(object? sender, EventArgs e)
03 {
04     elapsedTime++;
05     textBox1.Text = elapsedTime + "초 경과";
06     label1.Text = elapsedTime + "초 경과";
07 }
```

- 프로그램을 실행하면 1초마다 숫자가 증가하면서 프로그램이 시작된 지 몇 초나 지났는지 알려줌

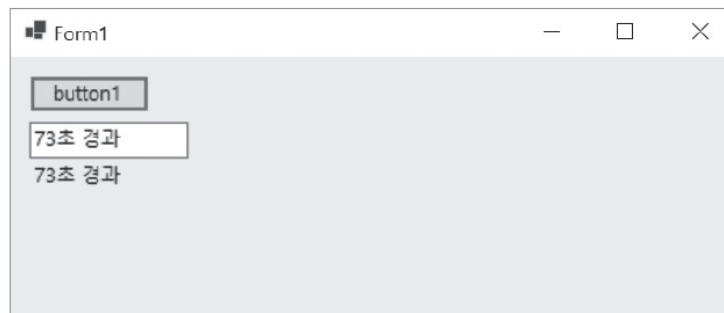


그림 6-38 프로그램이 시작된 지 몇 초가 지났는지 출력